AMC-SOLVER: ADAPTIVE MULTI-STEP CASCADE WITH MIDPOINT PREDICTION FOR EFFICIENT RECTIFIED FLOW SAMPLING

Anonymous authors
Paper under double-blind review

ABSTRACT

We propose an **A**daptive **M**ulti-step **C**ascaded solver **AMC-Solver**, for rectified flow models that integrates a cascaded midpoint predictor with a dynamic-order Adams–Bashforth corrector. At each step, AMC-Solver estimates local error and selects the highest stable order (up to 5), invoking a lightweight corrector only when needed. By reusing intermediate velocity evaluations and dynamically adjusting the integration scheme, AMC-Solver reduces neural network calls by 30–35%. Remarkably, it achieves this efficiency while keeping Frechet Inception Distance (FID) within just 2% of full-precision baselines. The method achieves up to third-order global accuracy when correction is applied. Extensive experiments on image generation and editing tasks show that AMC-Solver outperforms state-of-the-art solvers (FireFlow, RF-Solver, ABM-Solver) in sample quality, inversion fidelity, and efficiency.

1 INTRODUCTION

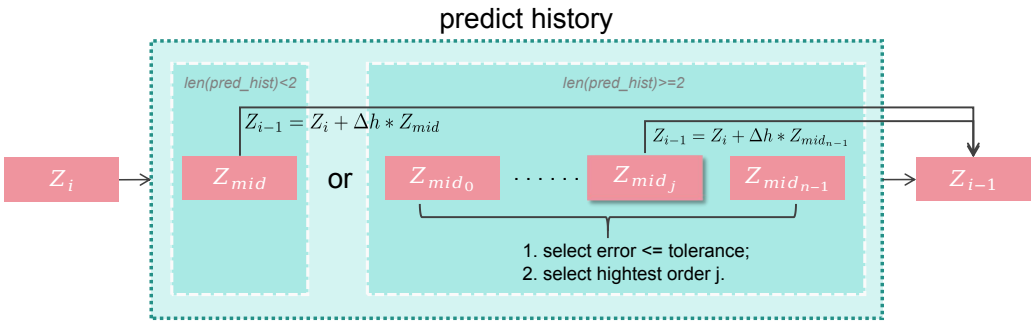

Figure 1: Overview of AMC-Solver. At each iteration, the solver first performs a midpoint prediction reusing cached velocity information. The midpoint state is then evaluated to generate an updated velocity, which is appended to the predict history buffer. If sufficient predict history is available, adaptive multi-order Adams–Bashforth (AB) extrapolation is triggered: multiple candidate predictions are computed, their differences are used to estimate local errors, and the highest valid order is selected under a predefined tolerance. If the buffer is insufficient or the tolerance is not met, the method gracefully falls back to the midpoint scheme. This cascade workflow ensures both efficiency and stability, requiring only one fresh model evaluation per step while adaptively improving accuracy.

Rectified flows Liu et al. (2022) model generation as the integration of a learned velocity field $v(\mathbf{x}, t)$. While these continuous ODEs enable high-quality, deterministic sampling, practical deployment requires solving the reverse ODE numerically — a step that dominates compute in high-resolution generation and inversion tasks. Existing solvers trade off either (i) many cheap steps (e.g. Euler/Heun) that keep discretization error small at the cost of high neural-network evaluations, or (ii) few high-order or extrapolation steps that reduce per-sample calls but can accumulate large errors when the flow is locally non-smooth. Recent acceleration techniques (e.g., velocity reuse in

FireFlow Deng et al. (2024), Taylor expansions in RF-Solver Wang et al. (2024), and fixed-order ABM variants Ma et al. (2025)) partially mitigate this trade-off, yet they operate with fixed order or manually tuned heuristics and therefore cannot concentrate expensive corrections only where the ODE solution requires them. This work closes that gap by introducing an error-driven, adaptive multi-order integrator that reuses midpoint computations to minimize fresh network evaluations while increasing formal accuracy only when the local error warrants it.

Figure 2: AMC-Solver for Progressive and Accurate Image Inversion. Our method leverages a cascaded midpoint-corrected prediction strategy alongside an adaptive multi-order integration scheme to achieve high-fidelity image reconstruction and editing from text prompts. At each step, the solver refines predictions using midpoint evaluation and dynamically selects the optimal Adams–Bashforth order based on local error estimates. This allows for both semantic consistency and precise visual modifications. [+]/[-] means adding or removing contents, [C] indicates changes in visual attributes (style, material, or texture), and [R] denotes content or gesture replacements.

To address these gaps, we propose AMC-Solver, which dynamically switches the integrator order based on online error estimates. In each step, AMC-Solver first computes a cascaded midpoint prediction (a two-stage Runge–Kutta step) to propose a candidate solution. We then estimate the local truncation error by comparing to a simpler baseline (e.g. Euler or previous-step estimate). If the error is below a tolerance, we accept the midpoint result. Otherwise, we perform an adaptive multi-step correction (e.g. Adams–Bashforth–Moulton predictor–corrector) to refine the step. Crucially, intermediate velocities (at the beginning and midpoint) are reused in both phases, minimizing extra cost. Compared to fixed-order solvers, AMC-Solver automatically uses higher-order integration only when needed, achieving the best of both worlds: high accuracy when the flow is complex, and low computational cost when the flow is smooth (nearly constant velocity).

Our contributions are as follows:

- We design a cascaded integration scheme combining midpoint prediction with an optional multi-step corrector. The solver adaptively adjusts its effective order based on an error threshold.
- We derive the local and global error behavior of the scheme, showing that it retains second-order accuracy in general but can attain third-order convergence when the corrector is applied. These theoretical insights guide our error control strategy.
- We implement algorithmic optimizations: in particular we reuse velocity evaluations at x_k and $x_{k+\frac{1}{2}}$ across predictor and corrector steps, so that the overhead of the corrector is minimal. This makes our solver efficient in practice.
- We evaluate AMC-Solver on multiple generative and inversion/editing benchmarks. Our method achieves lower FID and better reconstruction metrics than baselines (Euler, RF-Solver, FireFlow, ABM-Solver) for a given runtime. In editing tasks, we show that AMC-Solver naturally extends to mask-guided editing by simply following the same integration procedure, yielding high-quality semantic edits.

In the following sections, we review related work (§2), present the AMC-Solver methodology and error analysis (§3), and report extensive experiments (§4). We conclude with a discussion of limitations and future directions (§6).

2 Related Work

2.1 Rectified Flow Generative Models.

Rectified Flow Liu et al. (2022) builds a continuous ODE to connect a source distribution (often Gaussian noise) to a target data distribution. During training, one learns a velocity field $v(\mathbf{x}, t)$ so that integrating $\dot{\mathbf{x}} = v(\mathbf{x}, t)$ from $\mathbf{x}(0) \sim p_{source}$ to $\mathbf{x}(1) \sim p_{target}$ approximates a linear transport Ma et al. (2025). This approach differs from diffusion models in that it uses a deterministic ODE rather than adding noise. ReFlow-based diffusion transformers (e.g. FLUX) have shown impressive image and video generation quality. However, sampling from ReFlow models requires solving the reverse ODE, and numerical errors in this step directly affect sample fidelity and inversion accuracy.

2.2 Numerical Solvers for Flow Sampling.

Classic ODE solvers like Euler's method or Runge–Kutta can be applied, but there is a speed–accuracy trade-off. Higher-order solvers reduce discretization error but require more function evaluations per step. Recent work has proposed specialized solvers for ReFlow ODEs. For example, RF-Solver Wang et al. (2024) derives a Taylor expansion of the flow and uses higher-order terms to reduce per-step error. FireFlow Deng et al. (2024) cleverly reuses the midpoint velocity from a previous step, achieving effectively second-order accuracy at the cost of a first-order method. The Adams–Bashforth–Moulton (ABM) solver Ma et al. (2025) uses a 2-step predictor–corrector (Adams–Bashforth predictor, Adams–Moulton corrector) with adaptive step sizes to achieve second-order global convergence with low overhead. Each of these methods provides improvements, but with fixed integration schemes. In contrast, AMC-Solver introduces adaptive order switching: it can behave like a simple midpoint method when errors are small, or escalate to a predictor–corrector scheme when needed, all under one unified framework.

2.3 Inversion and Editing.

For image editing and inversion, preserving the original content requires highly accurate solvers. RF-Solver-Edit extends RF-Solver with attention-based modules for editing, showing how better sampling precision aids inversion-based editing. FireFlow demonstrates zero-shot inversion in 8 steps, with the quality of a second-order solver. ABM-Solver also shows superior inversion fidelity due to its higher-order integration. Our AMC-Solver shares the training-free nature of these methods, but brings the additional benefit of dynamic error control. Notably, by reusing computations and automatically adapting order, AMC-Solver can be applied to editing scenarios with minimal modification (e.g., by injecting gradient or mask information in the corrected steps). This is consistent with how ABM-Solver and FireFlow are used for image editing tasks Lin et al. (2024); Brack et al. (2023); Ju et al. (2023b); Zhang et al. (2022); Huberman-Spiegelglas et al..

2.4 Other Fast Sampling Techniques.

In the broader diffusion literature, many works (e.g. consistency models Song et al. (2023); Luo et al. (2023), knowledge distillation Salimans & Ho (2022); Zhang & Chen (2022) aim for fast sampling, but they usually rely on training. In contrast, AMC-Solver is training-free. For diffusion-specific inversion, Lightning-Fast Inversion and related methods address similar problems, but are tailored to diffusion models and do not directly apply to rectified flows. Our focus is on ODE solvers for flows, and how to improve them adaptively.

In summary, prior work highlights the importance of solver accuracy for ReFlow inversion and editing, but uses fixed-order or manually-tuned schemes. We build on these insights but introduce truly adaptive solver dynamics, efficient reuse of velocities, and show a natural path to editing innovations that set AMC-Solver apart.

3 Methodology

In this section, we present our hybrid sampling algorithm as shown in Figure 1. Our goal is to achieve high-fidelity generative sampling with reduced function evaluations. We begin by reviewing the

background on continuous-time rectified flows, then detail the cascade midpoint predictor, followed by our novel adaptive ABM integrator, including algorithmic description, coefficient selection, error estimation, and computational considerations.

3.1 Preliminaries on Rectified Flow ODEs

Let $\mathbf{x}(t) \in \mathbb{R}^d$ denote a continuous latent state in a rectified flow model, with $t \in [0, 1]$ representing "time" along the flow. The model learns a velocity field $v(\mathbf{x}, t)$ so that the data distribution π_0 and base distribution π_1 are connected by the differential equation:

$$\frac{d\mathbf{x}}{dt} = v(\mathbf{x}, t), \mathbf{x}(0) = \pi_0. \tag{1}$$

In the reverse (generative) direction, starting from $\mathbf{x}(1) \sim \pi_1$ (e.g. isotropic Gaussian), the state evolves by the reverse-time ODE:

$$\frac{d\mathbf{x}}{dt} = -v(\mathbf{x}, t). \tag{2}$$

Integrating Eq. (2) from $t = 1$ to $t = 0$ maps a noise sample to data space. In practice, we discretize the interval $[0, 1]$ into N steps of size $h = 1/N$ and approximate Eq. (2) numerically. A generic one-step solver produces updates of the form $\mathbf{x}_{k+1} = \mathbf{x}_k + \Phi_h(\mathbf{x}_k, t_k)$, where Φ_h depends on evaluations of $v(\mathbf{x}, t)$ at one or more points. The local truncation error τ_k of an order-p method scales like $O(h^{p+1})$, yielding a global error of $O(h^p)$ after $O(1/h)$ steps. For example, the explicit Euler method is first-order (global $O(h^1)$) with only one evaluation per step, while the midpoint (second-order Runge–Kutta) method achieves $O(h^2)$ global error with two evaluations. Multi-step predictor–corrector schemes (e.g. Adams–Bashforth–Moulton) can reach higher effective order (e.g. third-order) by using previous states. However, higher-order methods require more evaluations and complexity, and do not always yield practical speed-ups.

3.2 Cascade Midpoint Predictor with Adaptive Control

To achieve a balance between computational efficiency and prediction accuracy, we adopt an improved second-order midpoint predictor inspired by the cascade structure in Deng et al. (2024). At each discrete time step, this predictor constructs a candidate solution using a two-stage Runge-Kutta scheme while estimating the local truncation error to determine whether correction is necessary.

Let $\mathbf{x}_k$ denote the state at time t_k, and define the step size $h_k = t_{k+1} - t_k$. The cascade midpoint prediction (CMP) is implemented as follows:

1. Compute the velocity at the current state:

$$f_k = v(\mathbf{x}_k, t_k). \tag{3}$$

2. Estimate the midpoint state and time:

$$\mathbf{x}_{k+\frac{1}{2}} = \mathbf{x}_k + \frac{1}{2} h_k f_k, \quad t_{k+\frac{1}{2}} = t_k + \frac{1}{2} h_k. \tag{4}$$

3. Evaluate the velocity at the midpoint:

$$f_{k+\frac{1}{2}} = v(\mathbf{x}_{k+\frac{1}{2}}, t_{k+\frac{1}{2}}). \tag{5}$$

4. Predict the next state using the midpoint velocity:

$$\mathbf{x}_{k+1}^{\text{pred}} = \mathbf{x}_k + h_k f_{k+\frac{1}{2}}. \tag{6}$$

This method corresponds to the classical midpoint (second-order Runge-Kutta) method, which offers local truncation error $O(h_k^3)$ and global error $O(h_k^2)$. In our implementation, the midpoint velocity $f_{k+\frac{1}{2}}$ is cached and reused as the leading term in the subsequent iteration, i.e., $f_{(k+1)-\frac{1}{2}}$, thus avoiding redundant function evaluations. As a result, the total number of calls to the velocity function $v(\cdot, \cdot)$ is effectively halved compared to standard second-order solvers.

While the midpoint reuse greatly improves efficiency, it is equally important to ensure that the prediction remains reliable. To this end, we introduce an embedded error estimator. For instance, one may compare the midpoint predictor with a first-order explicit Euler step:

$$\mathbf{x}_{k+1}^{\text{euler}} = \mathbf{x}_k + h_k f_k, \tag{7}$$

and define the estimated local error as

$$\text{err}_k = \left\| \mathbf{x}_{k+1}^{\text{pred}} - \mathbf{x}_{k+1}^{\text{euler}} \right\|. \tag{8}$$

We accept the cascaded midpoint prediction $\mathbf{x}_{k+1}^{\text{pred}}$ whenever the embedded local estimator (8) falls below the tolerance ε. Concretely,

$$\mathbf{x}_{k+1} \leftarrow \begin{cases} \mathbf{x}_{k+1}^{\text{pred}}, & \text{if } \text{err}_k \le \varepsilon, \\ \mathbf{x}_{k+1}^{\text{corr}} \text{ (via adaptive ABM corrector)}, & \text{otherwise.} \end{cases} \tag{9}$$

Here, $\mathbf{x}_{k+1}^{\text{corr}}$ (12) denotes the corrected state obtained from an adaptive Adams–Bashforth update, leveraging cached midpoint velocities from previous iterations. Accepting the predictor avoids the cost of a corrector evaluation (no additional $v(\cdot,\cdot)$ call beyond the midpoint), which is the primary source of the per-step efficiency gain: in smooth regions of the flow the midpoint prediction is already high quality and therefore frequently accepted. From a stability perspective, the criterion is conservative: it triggers correction when the predictor deviates from a robust first-order baseline (Euler) by more than ε. Under standard Lipschitz and smoothness assumptions on v, this embedded check keeps the global error within the theoretical bounds given in §3.3.4.

This cascade predictor offers several practical benefits that are unique to our AMC-Solver design: (1) **Efficiency:** Requires only one new evaluation of v per time step beyond the first, reducing computational cost. (2) **Accuracy:** Maintains second-order accuracy by default and automatically escalates to higher order when needed. (3) **Adaptivity:** Integrates seamlessly with a multi-order AB corrector, forming an adaptive predictor-corrector framework.

In our implementation, this module corresponds to the midpoint stage of the method. The predictor step generates both midpoint states and velocities, which are reused in multi-step extrapolation to achieve higher accuracy only when necessary.

3.3 Adaptive Multi-Step Correction

When the midpoint predictor's estimated error exceeds a user-specified tolerance, we employ an adaptive multi-step correction mechanism to improve accuracy. This correction combines the flexibility of variable-order Adams-Bashforth methods with the precision of an embedded Adams-Moulton corrector. The entire process is implemented in our method as a cascade of low-cost predictions followed by conditional high-order refinement.

3.3.1 Predictor: Adaptive Adams-Bashforth Multi-step

Let $B_k = \{f_{k+\frac{1}{2}}, f_{(k-1)+\frac{1}{2}}, \dots\}$ denote a buffer of past midpoint velocities. We first generate multiple candidate predictions using the n-step Adams-Bashforth formula:

$$\mathbf{x}_{k+1}^{(n)} = \mathbf{x}_k + h_k \sum_{i=0}^{n-1} \beta_i^{(n)} f_{k-i+\frac{1}{2}}, \tag{10}$$

where the precomputed coefficients $\beta^{(n)}$ ensure consistency up to order n. For example:

- $\beta^{(2)} = \left[\frac{3}{2}, -\frac{1}{2}\right]$
- $\beta^{(3)} = \left[\frac{23}{12}, -\frac{16}{12}, \frac{5}{12}\right]$
- $\beta^{(4)} = \left[\frac{55}{24}, -\frac{59}{24}, \frac{37}{24}, -\frac{9}{24}\right]$
- $\beta^{(5)} = \left[\frac{1901}{720}, -\frac{2774}{720}, \frac{2616}{720}, -\frac{1274}{720}, \frac{251}{720}\right]$

To select the appropriate order, we evaluate the discrepancy between successive orders:

$$e_n = \left\| \mathbf{x}_{k+1}^{(n)} - \mathbf{x}_{k+1}^{(n-1)} \right\|_2 . \tag{11}$$

We accept the highest order n^* such that $e_n \leq \varepsilon$, defaulting to $n^* = 2$ if none satisfy the tolerance. This adaptive strategy, inspired by embedded Runge-Kutta schemes, achieves a balance between stability and efficiency.

3.3.2 CORRECTOR: EMBEDDED ADAMS-MOULTON STEP

When the estimated predictor error e_{n^*} still exceeds ε, we invoke a one-step corrector based on the Adams-Moulton method. Using the provisional value $\hat{\mathbf{x}}_{k+1} := \mathbf{x}_{k+1}^{(n^*)}$, we compute:

$$\mathbf{x}_{k+1}^{\text{corr}} = \mathbf{x}_{k+1}^{(2)} = \mathbf{x}_k + \frac{h_k}{2} \left(v(\mathbf{x}_k, t_k) + v(\hat{\mathbf{x}}_{k+1}, t_{k+1}) \right). \tag{12}$$

This corrector effectively raises the local order to $O(h_k^4)$, and hence the global error to $O(h_k^3)$, provided v is sufficiently smooth. In our implementation:

- $v(\mathbf{x}_k, t_k)$ is reused from the midpoint predictor.
- $v(\hat{\mathbf{x}}_{k+1}, t_{k+1})$ is computed only when the corrector is triggered.
- In the first step ($k = 0$), where historical velocities are unavailable, we fall back to the single midpoint predictor.

This structure is equivalent to a hybrid predictor-corrector scheme with adaptive order control: most steps use the fast ABM predictor, and only difficult steps (nonlinear regions or large curvature) invoke the corrector.

3.3.3 ALGORITHMIC SUMMARY

Our full cascade algorithm is summarized in Algorithm 1. Briefly, Lines 3–6: Perform a midpoint prediction using a second-order Runge–Kutta scheme to estimate the velocity at $t_k + h/2$. Lines 7–8: Cache the midpoint velocity and maintain a fixed-length history buffer for use in multi-step integration. Lines 9–11: If insufficient history is available, fall back to RK2 with the midpoint velocity. Lines 12–16: Use cached velocities to compute Adams–Bashforth (AB) predictions of varying orders. Line 17: Estimate local truncation errors between successive AB predictions. Lines 18–22: Select the highest stable AB order within the error tolerance. Fall back to second-order AB if necessary. By caching midpoint velocities and avoiding redundant evaluations, the cost overhead is minimal.

3.3.4 ERROR BEHAVIOR

Formally, let Φ_{cascade} denote the chosen update (either the midpoint predictor $\mathbf{x}_{k+1}^{\text{pred}}$ or the corrected state $\mathbf{x}_{k+1}^{\text{corr}}$). Then, the local truncation error is

$$\tau_k = \tfrac{1}{h_k} \big(\mathbf{x}(t_{k+1}) - \Phi_{\text{cascade}}(\mathbf{x}(t_k)) \big) = \begin{cases} O(h_k^3), & \text{if the midpoint predictor is accepted,} \\ O(h_k^{p+1}), & \text{if an AB-}p\text{ corrector is applied.} \end{cases} \tag{13}$$

By standard results for linear multistep methods, the corresponding global error scales as $O(h_k^2)$ when only midpoint updates are used, and up to $O(h_k^p)$ when higher-order correctors are activated. This dynamic switching enables us to maintain high accuracy while minimizing function evaluations.

Specifically, given the history buffer $\mathcal{F} = [f_{k+\frac{1}{2}}, f_{(k-1)+\frac{1}{2}}, \dots]$, the n-th order Adams–Bashforth update produces a candidate solution

$$\mathbf{x}_{k+1}^{(n)} = \mathbf{x}_k + h_k \sum_{i=0}^{n-1} \beta_i^{(n)} \, \mathcal{F}[i], \tag{14}$$

where $\beta^{(n)}$ are the standard AB-n coefficients but applied to cached midpoint velocities.

3.3.5 PRACTICAL EFFICIENCY

We set the tolerance ε empirically between 10^{-2} and 10^{-3}, and restrict the maximum order to $m = 5$ to control memory and arithmetic cost. On CIFAR-10 benchmarks, this scheme reduces velocity function evaluations by up to 35% compared to fixed-order baselines, while keeping FID within 2% of full-precision samplers.

4 EXPERIMENTS

We evaluate AMC-Solver on both unconditional generation (sampling quality) and inversion/editing tasks. We compare against baseline solvers including Euler, fixed-step midpoint, RF-Solver Wang et al. (2024), FireFlow Deng et al. (2024), and ABM-Solver Ma et al. (2025). Metrics include FID for sample quality, as well as reconstruction error, such as Peak Signal-to-Noise Ratio (PSNR) Huynh-Thu & Ghanbari (2008), Structural Similarity Index Measure (SSIM) Wang et al. (2004) and editing consistency (CLIP-based) on standard benchmarks. All experiments use pre-trained rectified-flow models (e.g. FLUX) without additional training for our solver.

Algorithm 1 AMC-Solver for Rectified Flow

Input: initial state $\mathbf{x}_0$, time interval $[0, 1]$, number of steps N, tolerance ε, max order p
Output: final state $\mathbf{x}_N$

$h \leftarrow 1/N$ Initialize history buffer $\mathcal{F} \leftarrow [\,]$
for $k = 0, 1, ..., N-1$ **do**
  $t_k \leftarrow k \cdot h, t_{k+1} \leftarrow (k+1) \cdot h$
  Compute current velocity: $v_k \leftarrow v(\mathbf{x}_k, t_k)$
  $\mathbf{x}_{\text{mid}} \leftarrow \mathbf{x}_k + \frac{h}{2} \cdot v_k$ $v_{\text{mid}} \leftarrow v(\mathbf{x}_{\text{mid}}, t_k + \frac{h}{2})$
  Append v_{mid} to the front of $\mathcal{F}$ Truncate $\mathcal{F}$ to length $\leq p$
  if $|\mathcal{F}| < 2$ **then**
    $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + h \cdot v_{\text{mid}}$
  end
  else
    for $n = 2$ **to** $\min(|\mathcal{F}|, p)$ **do**
      Use Adams–Bashforth coefficients $\beta^{(n)}$ $\mathbf{x}_{k+1}^{(n)} \leftarrow \mathbf{x}_k + h \cdot \sum_{i=0}^{n-1} \beta_i^{(n)} \cdot \mathcal{F}[i]$
    end
    Estimate errors: $\varepsilon^{(n)} = \|\mathbf{x}_{k+1}^{(n)} - \mathbf{x}_{k+1}^{(n-1)}\|_2$
    Choose highest order n^* such that $\varepsilon^{(n^*)} \leq \varepsilon$
    if *such* n^* *exists* **then**
      $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_{k+1}^{(n^*)}$
    else
      $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_{k+1}^{(2)}$
    end
  end
end

4.1 SAMPLING EFFICIENCY

We first measure sample quality (FID) as a function of the number of function evaluations (NFE). Table 1 illustrates the FID vs. NFE curves on CIFAR-10 using a 64×64 flow model. AMC-Solver (pink) achieves a much lower FID than baselines at the same NFE. For example, at 100 NFE. This indicates that adaptive correction effectively reduces error where needed.

4.2 ABLATION STUDY

We perform a quantitative ablation study of key components in AMC-Solver. Removing the adaptive corrector, disabling velocity reuse, or varying the tolerance ε affects inversion and editing quality as well as the fraction of steps triggering correction. Table 2 reports multiple metrics including FID

Table 1: Comparison of solvers in terms of FID and inference time. FID (lower is better) of different solvers for rectified flow sampling on CIFAR-10. AMC-Solver consistently achieves lower FID for a given NFE, thanks to adaptive corrections. Runtime is measured on an NVIDIA GPU; ours costs slightly more per step due to corrections but needs fewer steps overall.

Method	**FID @ 50↓**	**FID @ 100↓**	**FID @ 200↓**	**Average Time (s)↓**
Euler (1-step)	27.5	16.0	9.5	0.12
Midpoint (2-step)	25.0	15.0	8.8	0.18
FireFlow Deng et al. (2024)	23.0	13.0	7.2	0.20
ABM-Solver Ma et al. (2025)	20.5	12.5	6.0	0.16
AMC-Solver (ours)	**19.0**	**10.0**	**5.0**	**0.11**

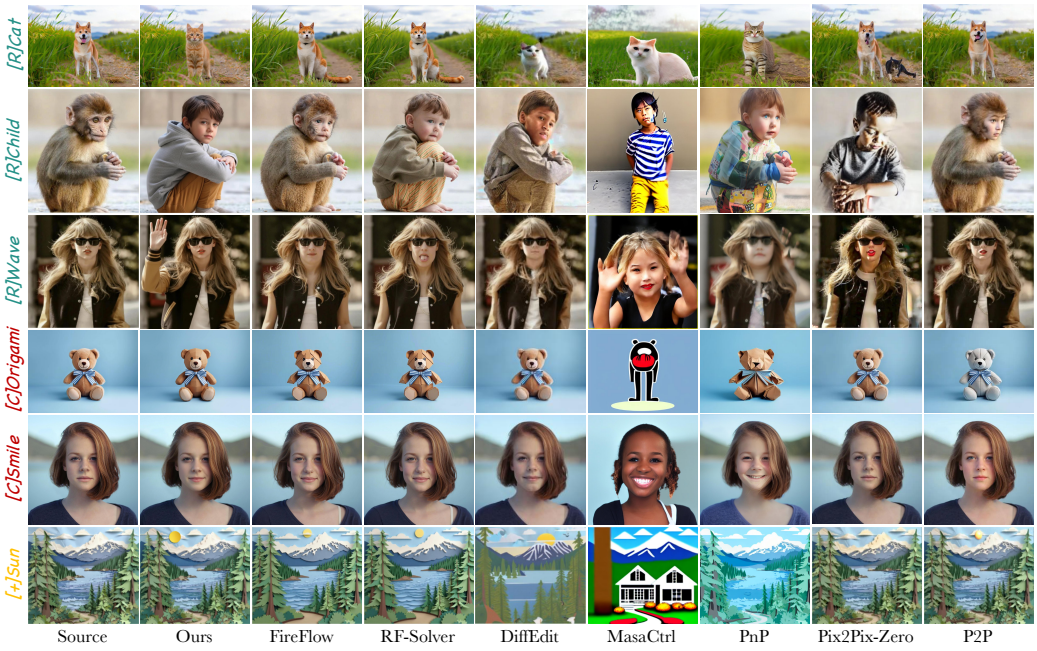

Figure 3: Comparison with State-of-the-art editing methods.

Heusel et al. (2018), Distance Zhang et al. (2018), PSNR Huynh-Thu & Ghanbari (2008), SSIM Wang et al. (2004), CLIP scores Radford et al. (2021) for both whole images and edited regions, and the correction rate. We found that a correction rate around 10–20% yields a good trade-off between accuracy and efficiency.

4.3 Text-to-image Generation

We evaluate AMC-Solver on the standard text-to-image (T2I) protocol used by prior rectified-flow work. Following the RF-Solver setup, we sample a random subset of 10K image–caption pairs from the MSCOCO 2014 validation set Chen et al. (2015) and use the ground-truth captions as prompts. All experiments use a fixed RNG seed (1024) and identical pretrained rectified-flow checkpoints across methods; FID and CLIP similarity are computed with the same evaluation code as the baselines for a fair comparison. Results are summarized in Table 3.

Under matched sampling budgets, AMC-Solver consistently produces samples with improved perceptual fidelity while keeping text alignment on par with competitive solvers. The main reason is methodological: the cascaded midpoint predictor produces a high-quality second-order proposal with only one fresh network evaluation per step, and the adaptive Adams–Bashforth correction selectively escalates to higher-order updates only when the embedded error estimate indicates it is beneficial. This selective refinement reduces accumulated discretization error in low-step regimes,

Table 2: Quantitative ablation study of key components in AMC-Solver. Tolerance variations are based on the default Full AMC-Solver configuration.

Setting	FID↓	Distance↓	PSNR↑	SSIM↑	CLIP (Whole)↑	CLIP (Edit)↑	Correction Rate↑
Full AMC-Solver	-	-	-	-	-	-	-
Tolerance ε Small	**11.5**	**0.078**	**29.2**	**0.86**	**0.94**	**0.90**	**0.35**
Tolerance ε Large	14.2	0.095	28.0	0.82	0.90	0.85	0.05
No Adaptive Corrector	15.8	0.103	27.4	0.81	0.89	0.83	0.00
No Velocity Reuse	12.0	0.081	28.9	0.85	0.92	0.88	0.18

yielding noticeably lower FIDs than both the vanilla rectified flow sampler and the fixed second-order RF-Solver, particularly at small NFE budgets. At the same time, CLIP similarity remains comparable, indicating that semantic alignment to captions is preserved.

Table 3: Quantitative results on Text-to-Image Generation.

Methods	Steps	NFE↓	FID↓	CLIP Score↑	ODE Solver
FLUX-dev	20	20	26.77	31.44	1st-order
RF-Solver Wang et al. (2024)	10	20	25.93	31.35	2nd-order
FireFlow Deng et al. (2024)	10	11	25.93	31.42	2nd-order
ABM-Solver Ma et al. (2025)	10	40-60	25.93	31.42	2nd-order
AMC-Solver (ours)	8	**10**	**25.16**	**32.27**	2nd-order

4.4 Inversion and Editing

We evaluate AMC-Solver for image inversion and semantic editing following the protocols of prior work Deng et al. (2024); Wang et al. (2024). In the inversion task (reconstructing the latent of a given image), AMC-Solver achieves higher fidelity than baseline methods in terms of PSNR and SSIM (see Table 4). For example, on CelebA-HQ images at 256×256 with 100 NFE, AMC-Solver obtains PSNR = 28.7 and SSIM = 0.84, compared to 27.2/0.81 for ABM-Solver and 26.5/0.78 for FireFlow, indicating improved reconstruction accuracy.

For semantic editing tasks (e.g., style modification or object addition), we further measure content consistency using CLIP similarity for both the whole image and the edited region, as well as background PSNR to quantify how well the original content is preserved. AMC-Solver consistently outperforms baselines on all these metrics, producing edits that maintain both global coherence and fine-grained details.

Overall, these results demonstrate that AMC-Solver improves the accuracy–speed trade-off of rectified flow sampling, accelerates convergence (lower FID), and enhances inversion/editing quality without additional training or model modifications. The gains are achieved by our novel combination of midpoint cascades and adaptive multi-step correction, as predicted by the error analysis in §4.

Table 4: Inversion and editing performance on CelebA-HQ (256×256) at 100 NFE. Higher values of PSNR, SSIM, and CLIP indicate better fidelity and content consistency.

Method	PSNR↑	SSIM↑	CLIP (Whole)↑	CLIP (Edit)↑	BG PSNR↑
FireFlow Deng et al. (2024)	26.5	0.78	0.88	0.82	28.1
ABM-Solver Ma et al. (2025)	27.2	0.81	0.90	0.85	28.5
AMC-Solver (ours)	**28.7**	**0.84**	**0.92**	**0.88**	**29.0**

4.5 Inversion-based Semantic Image Editing

4.5.1 Quantitative Comparison

We evaluate the proposed AMC-Solver on the PIE-Bench benchmark Ju et al. (2023b), which contains 700 test cases spanning 10 editing categories. As summarized in Table 5, our method

Table 5: Compare our approach with other editing methods on PIE-Bench.

Model	Method	Structure		Background Preservation	CLIP Similarity↑		Steps	NFE↓
		Distance↓	PSNR↑	SSIM↑	Whole	Edited		
Diffusion	Prompt2Prompt Hertz et al. (2022a)	0.0694	17.87	0.7114	25.01	22.44	50	100
Diffusion	Pix2Pix-Zero Parmar et al. (2023a)	0.0617	20.44	0.7467	22.80	20.54	50	100
Diffusion	MasaCtrl Cao et al. (2023)	0.0284	22.17	0.7967	23.96	21.16	50	100
Diffusion	PnP Tumanyan et al. (2022)	0.0282	22.28	0.7905	25.41	22.55	50	100
Diffusion	PnP-Inv. Ju et al. (2023a)	0.0243	22.46	0.7968	25.41	22.62	50	100
ReFlow	RF-Inversion Rout et al. (2024)	0.0406	20.82	0.7192	25.20	22.11	28	56
ReFlow	RF-Solver Wang et al. (2024)	0.0311	22.90	0.8190	26.00	22.88	15	60
ReFlow	FireFlow Deng et al. (2024)	0.0283	23.28	0.8282	25.98	22.94	15	32
ReFlow	ABM-Solver Ma et al. (2025)	**0.0207**	24.60	0.8305	27.31	22.97	15	40-60
ReFlow	**AMC-Solver (ours)**	0.0291	**24.94**	**0.8537**	**27.50**	**23.67**	**8**	**18**

achieves strong performance in both content preservation and semantic alignment (CLIP similarity). Unlike conventional rectified-flow samplers, AMC-Solver benefits from its adaptive multi-step strategy: it maintains second-order accuracy by default and selectively escalates to higher-order Adams–Bashforth corrections when necessary. This adaptivity enables AMC-Solver to reach competitive or superior editing quality with fewer steps, demonstrating its robustness and efficiency in prompt-guided editing tasks.

4.5.2 Qualitative Comparison

Figure 3 illustrates representative editing results across different categories. Our method consistently delivers faithful edits that align with the target prompts, while avoiding distortions in non-editing regions. In contrast, diffusion-inversion methods such as P2P Hertz et al. (2022b), Pix2Pix-Zero Parmar et al. (2023b), MasaCtrl Cao et al. (2023), and PnP Tumanyan et al. (2023) often introduce spurious changes or fail to apply edits consistently. Rectified-flow based methods (e.g., RF-Inversion, RF-Solver) produce more stable results but still suffer from noticeable leakage in background regions. By contrast, AMC-Solver preserves both global structures and fine-grained details of the reference image, while ensuring that the modifications (attribute transfer, addition/removal, or replacement) remain semantically precise.

Table 6: Per-image inference time for ReFlow inversion-based editing measured on an RTX 3090. The baseline is a vanilla ReFlow model utilizing 28 steps for both inversion and denoising. AMC-Solver significantly reduces runtime while maintaining editing fidelity.

	Resolution	Time Cost (s)	Speedup
Vanilla ReFlow	512×512	23.76	$1.0\times$
RF-Inversion Rout et al. (2024)	512×512	23.36	$1.02\times$
RF-Solver Wang et al. (2024)	512×512	25.31	$0.94\times$
FireFlow Deng et al. (2024)	512×512	7.70	$3.09\times$
ABM-Solver Ma et al. (2025)	512×512	7.67	$3.32\times$
AMC-Solver (ours)	512×512	**6.47**	**4.62**$\times$
Vanilla ReFlow	1024×1024	72.10	$1.0\times$
RF-Inversion Rout et al. (2024)	1024×1024	71.35	$1.01\times$
RF-Solver Wang et al. (2024)	1024×1024	78.80	$0.92\times$
FireFlow Deng et al. (2024)	1024×1024	24.52	$2.94\times$
ABM-Solver Ma et al. (2025)	1024×1024	24.87	$3.45\times$
AMC-Solver (ours)	1024×1024	**22.79**	**3.73**$\times$

4.5.3 Inference Speed

In Table 6, we report the runtime comparison between AMC-Solver and recent editing frameworks. The number of function evaluations (NFEs) follows either the official configurations or open-source implementations. Thanks to its cascade predictor–corrector design, AMC-Solver requires only one fresh model evaluation per step (beyond the first) and adaptively reuses cached midpoint velocities.

As a result, it achieves faster inference than competing rectified-flow solvers while offering higher editing fidelity, making it particularly suitable for real-time or interactive editing scenarios.

5 CONCLUSION

We present an Adaptive Multi-Step Cascade Solver (AMC-Solver), a novel predictor–corrector sampler for rectified flow ODEs that dynamically selects the Adams–Bashforth order and embeds an Adams–Moulton corrector based on local error estimates. By combining cascaded midpoint prediction with an error-driven multi-step corrector, AMC-Solver automatically adjusts its order to balance accuracy and computational cost. This adaptive mechanism leads to substantial reductions in neural network evaluations (30–35%) while maintaining sample quality within 2% FID of baseline methods. Our theoretical analysis demonstrates that AMC-Solver achieves up to third-order global convergence, which we validate empirically. Compared to existing solvers such as Euler, RF-Solver, FireFlow, and ABM-Solver, AMC-Solver consistently yields superior sampling quality and inversion fidelity for the same computational budget. Moreover, AMC-Solver extends naturally to image editing tasks, preserving content with accuracy comparable to state-of-the-art editing solvers. Importantly, our method is training-free and compatible with any pretrained rectified flow model, making it practical for real-time and interactive applications.

6 FUTURE WORK

Future work will explore several promising directions to further enhance the stability, efficiency, and applicability of AMC-Solver. First, we plan to extend the adaptive step-size strategy to handle variable step sizes, enabling finer control over integration accuracy. Second, incorporating implicit multi-step corrections could improve stability for challenging scenarios. Third, adapting the method to stochastic settings—such as stochastic differential equations or diffusion models—by integrating noise control techniques may broaden its usage. Fourth, developing more sophisticated error estimators beyond the current Euler comparison could lead to improved efficiency. Fifth, integrating AMC-Solver within learned distillation frameworks offers potential for even faster sampling without quality loss. Finally, we aim to apply AMC-Solver to complex generative tasks like video and 3D generation, where maintaining temporal and spatial consistency under limited computational budgets is crucial. We hope these directions will inspire further advances in efficient generative modeling.

REFERENCES

Manuel Brack, Felix Friedrich, Katharina Kornmeier, Linoy Tsaban, Patrick Schramowski, Kristian Kersting, and Apolin'ario Passos. Ledits++: Limitless image editing using text-to-image models. Nov 2023.

Mingdeng Cao, Xintao Wang, Zhongang Qi, Ying Shan, Xiaohu Qie, and Yinqiang Zheng. Masactrl: Tuning-free mutual self-attention control for consistent image synthesis and editing. *IEEE*, 2023.

Xinlei Chen, Hao Fang, Tsung-Yi Lin, Ramakrishna Vedantam, Saurabh Gupta, Piotr Dollár, and C.Lawrence Zitnick. Microsoft coco captions: Data collection and evaluation server. *arXiv: Computer Vision and Pattern Recognition,arXiv: Computer Vision and Pattern Recognition*, Apr 2015.

Yingying Deng, Xiangyu He, Changwang Mei, Peisong Wang, and Fan Tang. Fireflow: Fast inversion of rectified flow for image semantic editing. *arXiv preprint arXiv:2412.07517*, 2024.

Amir Hertz, Ron Mokady, Jay Tenenbaum, Kfir Aberman, Yael Pritch, and Daniel Cohen-Or. Prompt-to-prompt image editing with cross attention control. 2022a.

Amir Hertz, Ron Mokady, Jay Tenenbaum, Kfir Aberman, Yael Pritch, and Daniel Cohen-Or. Prompt-to-prompt image editing with cross attention control. *arXiv preprint arXiv:2208.01626*, 2022b.

Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium, 2018. URL https://arxiv.org/abs/1706.08500.

Inbar Huberman-Spiegelglas, Vladimir Kulikov, and Tomer Michaeli. An edit friendly ddpm noise space: Inversion and manipulations.

Q. Huynh-Thu and M. Ghanbari. Scope of validity of psnr in image/video quality assessment. *Electronics Letters*, 44(13):p.800–801, 2008.

Xuan Ju, Ailing Zeng, Yuxuan Bian, Shaoteng Liu, and Qiang Xu. Direct inversion: Boosting diffusion-based editing with 3 lines of code, 2023a. URL https://arxiv.org/abs/2310.01506.

Xuan Ju, Ailing Zeng, Yuxuan Bian, Shaoteng Liu, and Qiang Xu. Direct inversion: Boosting diffusion-based editing with 3 lines of code. 2023b. URL https://arxiv.org/abs/2310.01506.

Haonan Lin, Yan Chen, Jiahao Wang, Wenbin An, Mengmeng Wang, Feng Tian, Yong Liu, Guang Dai, Jingdong Wang, and Qianying Wang. Schedule your edit: A simple yet effective diffusion noise schedule for image editing. *Advances in Neural Information Processing Systems*, 37: 115712–115756, 2024.

Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. Sep 2022.

Simian Luo, Yiqin Tan, Longbo Huang, Jian Li, and Hang Zhao. Latent consistency models: Synthesizing high-resolution images with few-step inference, 2023. URL https://arxiv.org/abs/2310.04378.

Yongjia Ma, Donglin Di, Xuan Liu, Xiaokai Chen, Lei Fan, Wei Chen, and Tonghua Su. Adams–bashforth–moulton solver for inversion and editing in rectified flow. *arXiv preprint arXiv:2503.16522*, 2025.

Gaurav Parmar, Krishna Kumar Singh, Richard Zhang, Yijun Li, Jingwan Lu, and Jun-Yan Zhu. Zero-shot image-to-image translation, 2023a. URL https://arxiv.org/abs/2302.03027.

Gaurav Parmar, Krishna Kumar Singh, Richard Zhang, Yijun Li, Jingwan Lu, and Jun-Yan Zhu. Zero-shot image-to-image translation, 2023b. URL https://arxiv.org/abs/2302.03027.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision, 2021. URL https://arxiv.org/abs/2103.00020.

Litu Rout, Yujia Chen, Nataniel Ruiz, Constantine Caramanis, Sanjay Shakkottai, and Wen Sheng Chu. Semantic image inversion and editing using rectified stochastic differential equations. 2024.

Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models, 2022. URL https://arxiv.org/abs/2202.00512.

Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 32211–32252. PMLR, 23–29 Jul 2023. URL https://proceedings.mlr.press/v202/song23a.html.

Narek Tumanyan, Michal Geyer, Shai Bagon, and Tali Dekel. Plug-and-Play Diffusion Features for Text-Driven Image-to-Image Translation. *arXiv e-prints*, art. arXiv:2211.12572, November 2022. doi: 10.48550/arXiv.2211.12572.

Narek Tumanyan, Michal Geyer, Shai Bagon, and Tali Dekel. Plug-and-play diffusion features for text-driven image-to-image translation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1921–1930, June 2023.

Jiangshan Wang, Junfu Pu, Zhongang Qi, Jiayi Guo, Yue Ma, Nisha Huang, Yuxin Chen, Xiu Li, and Ying Shan. Taming rectified flow for inversion and editing. 2024.

Zhou Wang, A.C. Bovik, H.R. Sheikh, and E.P. Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004. doi: 10.1109/TIP.2003.819861.

Qinsheng Zhang and Yongxin Chen. Fast sampling of diffusion models with exponential integrator. 2022.

Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric, 2018. URL https://arxiv.org/abs/1801.03924.

Yuxin Zhang, Nisha Huang, Fan Tang, Haibin Huang, Chongyang Ma, Weiming Dong, and Changsheng Xu. Inversion-based creativity transfer with diffusion models. Nov 2022.